

MEDIATEK

MT8676 Yocto Audio User Manual

Version: 1.0
Release date: 2024-08-12

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

Version History

Version	Date	Author	Description
1.0	2024-08-12	Qimei Liu	Official release

Table of Contents

Version History	2
Table of Contents	3
List of Figures	3
List of Tables	3
1 Audio	4
1.1 Overview	4
1.1.1 Brief Introduction	4
1.1.2 Abbreviation	4
1.2 Architecture/Process Overview	4
1.2.1 Hardware	4
1.2.2 Software	8
1.2.3 Audio Driver	8
1.3 Configuration/Customization Guideline	13
1.3.1 Kernel DTS	13
1.3.2 Audio Device Configuration	14
1.4 Frequently Asked Questions/Troubleshooting	15
1.4.1 How to Test Audio Driver?	15
1.4.2 How to Enable Audio Log?	16
Exhibit 1 Terms and Conditions	17

List of Figures

Figure 1-1. Audio system	5
Figure 1-2. AFE block diagram	7
Figure 1-3. Yocto audio system	8
Figure 1-4. Audio driver architecture	9
Figure 1-5. Audio driver components	10
Figure 1-6. Audio devices	10
Figure 1-7. Audio CPU DAI	11
Figure 1-8. DPCM devices	12
Figure 1-9. Playback & capture devices	13
Figure 1-10. MTK Audio DTS	14
Figure 1-11. Audio path for playing music	15

List of Tables

Table 1-1. Abbreviation	4
-------------------------------	---

1 Audio

1.1 Overview

1.1.1 Brief Introduction

This chapter introduces the hardware, software, and basic functionalities of the MT8676 Audio system.

1.1.2 Abbreviation

Table 1-1. Abbreviation

Abbreviation	Explanation
AFE	Audio Front End
DAI	Digital Audio Interface
DTS	Device Tree Source
I2S	Integrated Interchip Sound (I2S) is a digital audio transmission standard defined by Philips in 1986 and revised in 1996. It is used for the transmission of digital audio data between devices.
PCM	Pulse Code Modulation (PCM) is a method used to digitally represent analog signals. In PCM, the amplitude of the analog signal is sampled at uniform intervals, and each sample is quantized to the nearest value within a range of digital steps. This process converts the analog signal into a digital format that can be easily transmitted and processed by digital systems.
TDM	Time Division Multiplexing (TDM) is a method of transmitting and receiving multiple signals over a common communication channel by dividing the time frame into several time slots. Each signal is assigned a specific time slot during which it can transmit its data. This allows multiple signals to share the same transmission medium while maintaining their individual data integrity.

1.2 Architecture/Process Overview

1.2.1 Hardware

The overall hardware composition of the MT8676 Audio System is illustrated in [Figure 1-1](#):

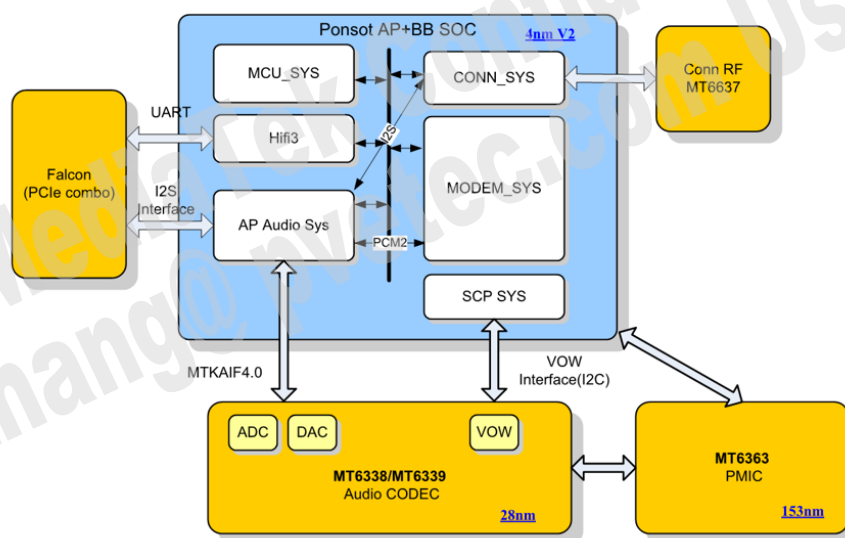
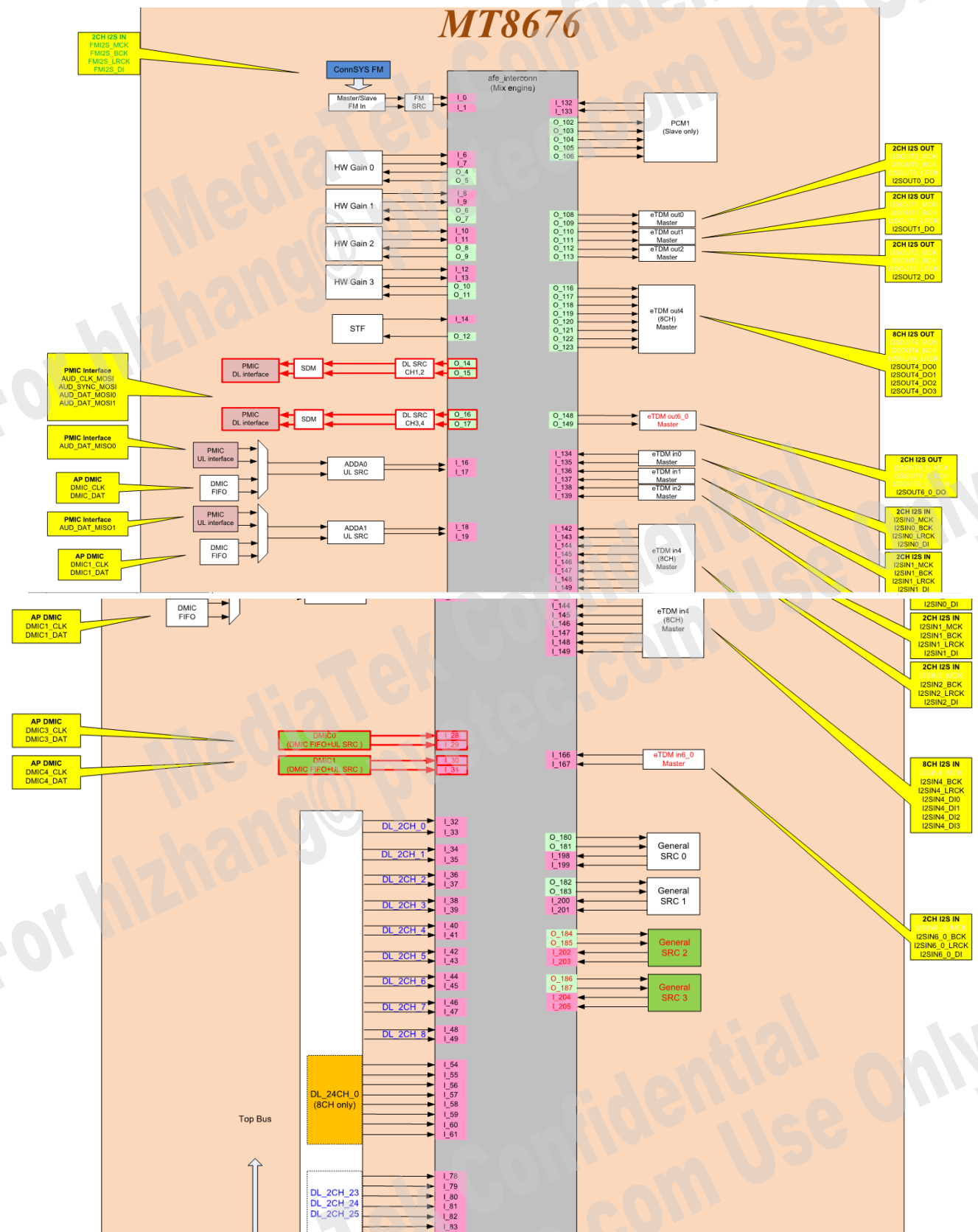


Figure 1-1. Audio system

The audio hardware includes external hardware interfaces, AFE_Interconn, AFE_Tinyconn, and AFE_Memif, as well as the backend Codec 6338. The external hardware interfaces are as follows:

- Master I2S output *5
- Master I2S input *4
- Master TDM output *1
- Master TDM input *1
- Slave PCM interface for internal modem *1
- Proprietary audio interface for MTK PMIC codec *1
- Slave I2S input interface with SRC for connsys FM *1
- PDM interface for DMIC *2



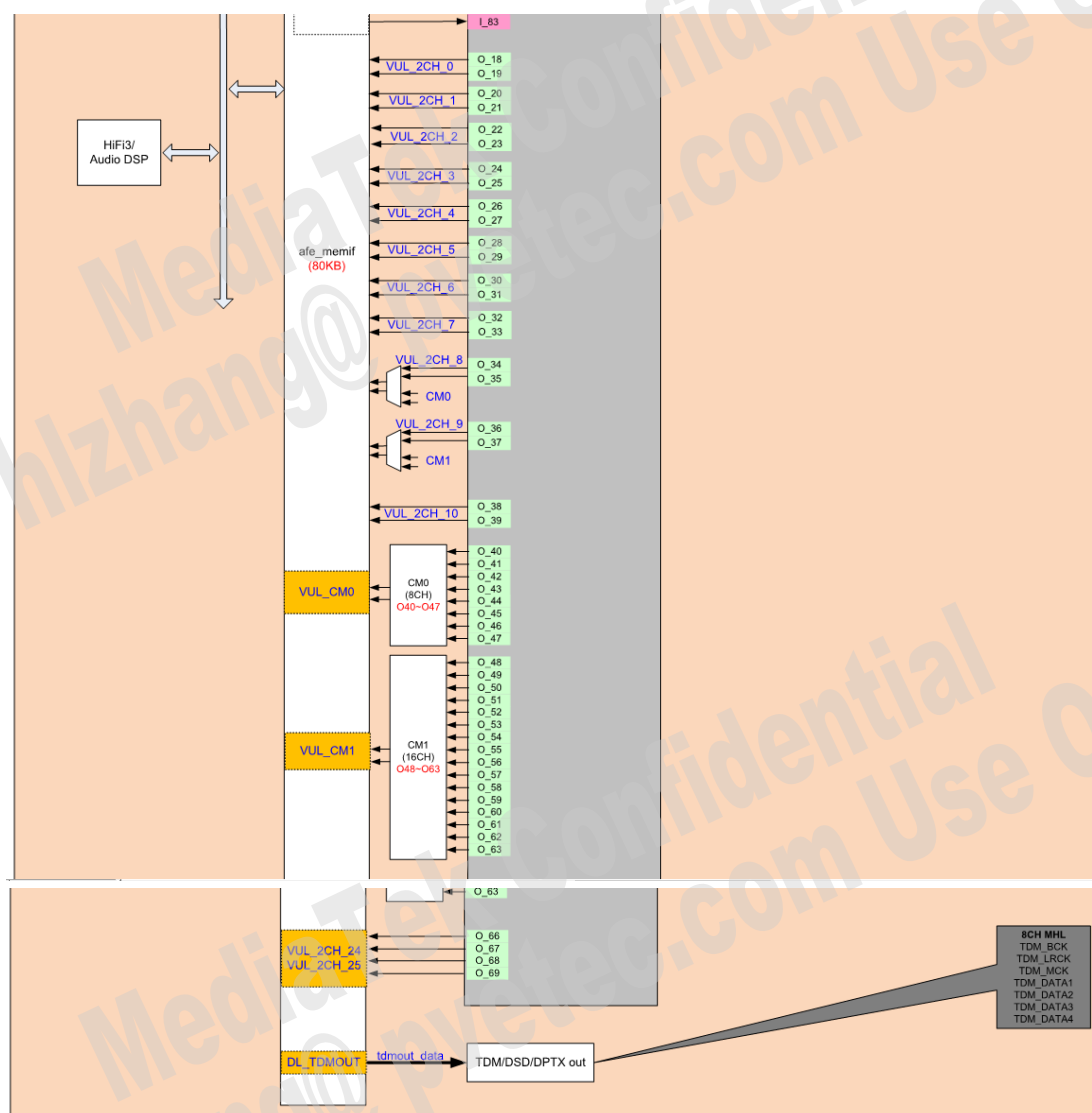


Figure 1-2. AFE block diagram

The internal AFE hardware interfaces can be categorized into two types:

- **Memory Interfaces:**

This is known as AFE_memif, this part functions similarly to DMA. It is primarily responsible for transferring PCM data from upper-layer applications to the specified downlink (DL) port for playback or capturing PCM data from the specified uplink (UL) port and transmitting it to upper-layer applications.

- **Connect Interfaces:**

There are two types: AFE_interconn and AFE_tiny_conn. The main difference between them is the supported data bit width. AFE_interconn supports up to 24-bit data, while AFE_tiny_conn can support up to 32-bit data. Both serve the same purpose, which is to connect the DL or UL of Memif with backend hardware interfaces like I2S/TDM, creating an I/O path for data transmission with external devices. For playback, this means sending DL data to the specified output port (e.g., I2S/TDM out). Conversely, for capture, it means connecting the UL interface to the specified input port (e.g., I2S/TDM in) to capture data from external devices. Additionally, HW SRC, gain control, and mix functions can be added to the I/O path.

1.2.2 Software

The software architecture of the MT8676 Yocto Audio System is illustrated in Figure 1-3:

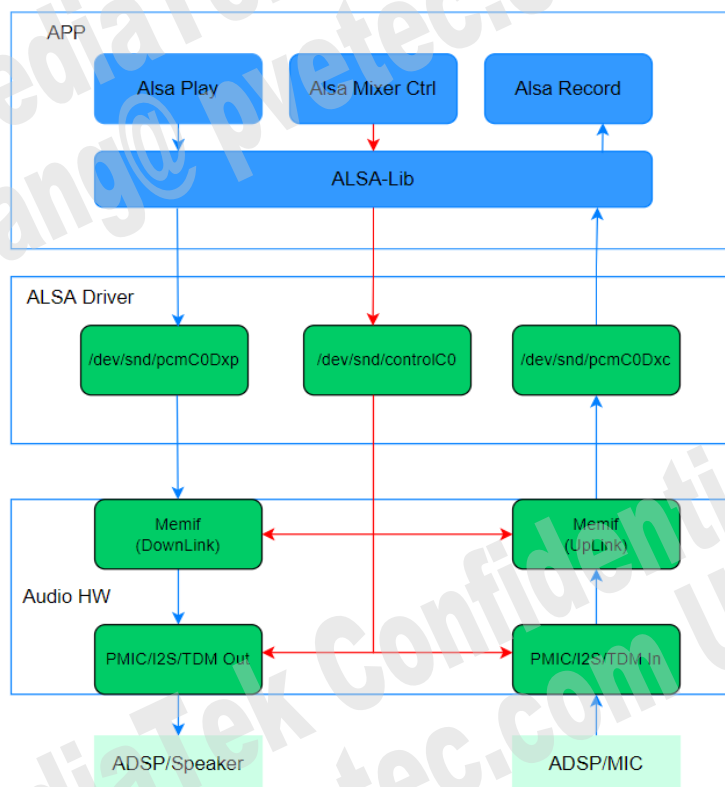


Figure 1-3. Yocto audio system

The ALSA Lib module is the native audio module for Linux. Detailed information about this module can be found online, so it will not be elaborated upon here. This document primarily focuses on explaining the proprietary MTK Audio Driver.

1.2.3 Audio Driver

Audio driver is based on the standard ALSA ASoC architecture:

- **Machine:** Refers to a specific machine or development board. It is evident that the Machine is almost non-reusable, as the hardware implementation on each Machine can be different—different CPUs, different Codecs, and different audio input and output devices. The Machine provides a carrier for the CPU, Codec, and input/output devices. The MT8676 reference board is one such Machine, and a customized development board is also a Machine.
- **Platform:** Generally, refers to a specific SoC platform, such as the MT8676. Audio-related components usually include the clock, DMA, I2S, PCM, etc., within the SoC. Once the SoC is specified, we can assume it will have a corresponding Platform, which is related only to the SoC and not to the Machine. The AFE mentioned above belongs to the Platform.
- **Codec:** Generally, refers to I2S/TDM interfaces, D/A, A/D, mixers, Power Amplifiers (PA), etc. It usually includes multiple inputs (Mic, Line-in, I2S, PCM) and multiple outputs (headphones, speakers, earpiece, Line-out). The Codec is also a

reusable component; the same Codec can be used by different Machines. The Codec corresponding to the MT8676 reference board is the PMIC 6338, and the external DSP used by customers is also considered as a Codec.

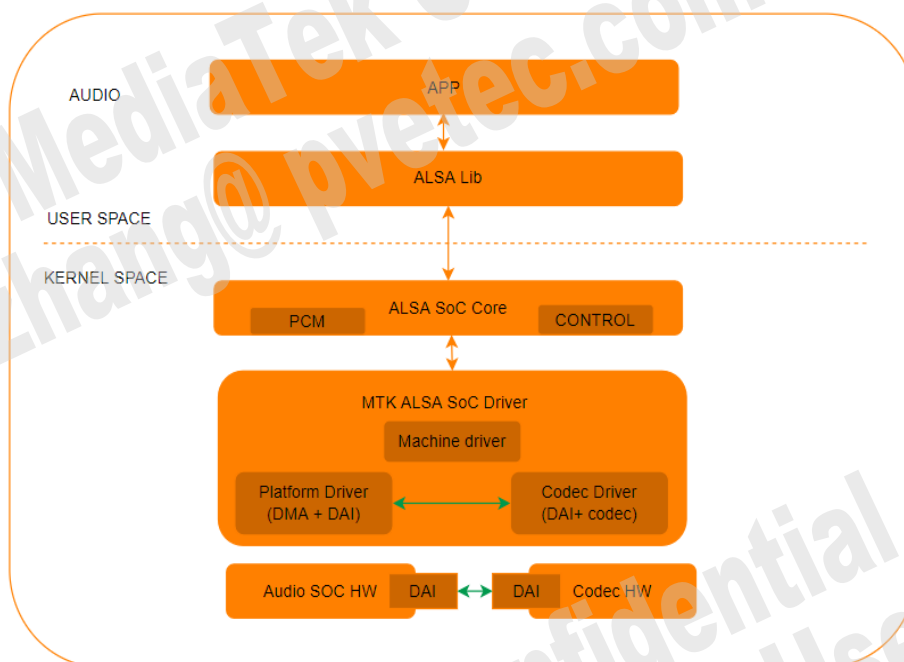


Figure 1-4. Audio driver architecture

Corresponding drivers for the above three components:

- **Machine driver:** Responsible for handling machine-specific controls and audio events (for example, turning on an amplifier when playing audio). The Platform and Codec drivers cannot work independently; they must be combined by the Machine driver to complete the audio processing of the entire device.
- **Platform driver:** Contains the configuration and control of the audio DMA and audio interfaces (I2S, PCM, etc.) for the SoC platform. It must not include any code related to the board or machine.
- **Codec driver:** A key design principle in ASoC is that the Codec driver must be platform-independent. It includes some audio controls, audio interfaces, definitions for DAPM (Dynamic Audio Power Management), and certain Codec IO functions. To ensure hardware independence, any platform-specific and machine-specific code must be moved to the Platform and Machine drivers.

The Audio Driver modules corresponding to the MT8676 platform are shown as [Figure 1-5](#):

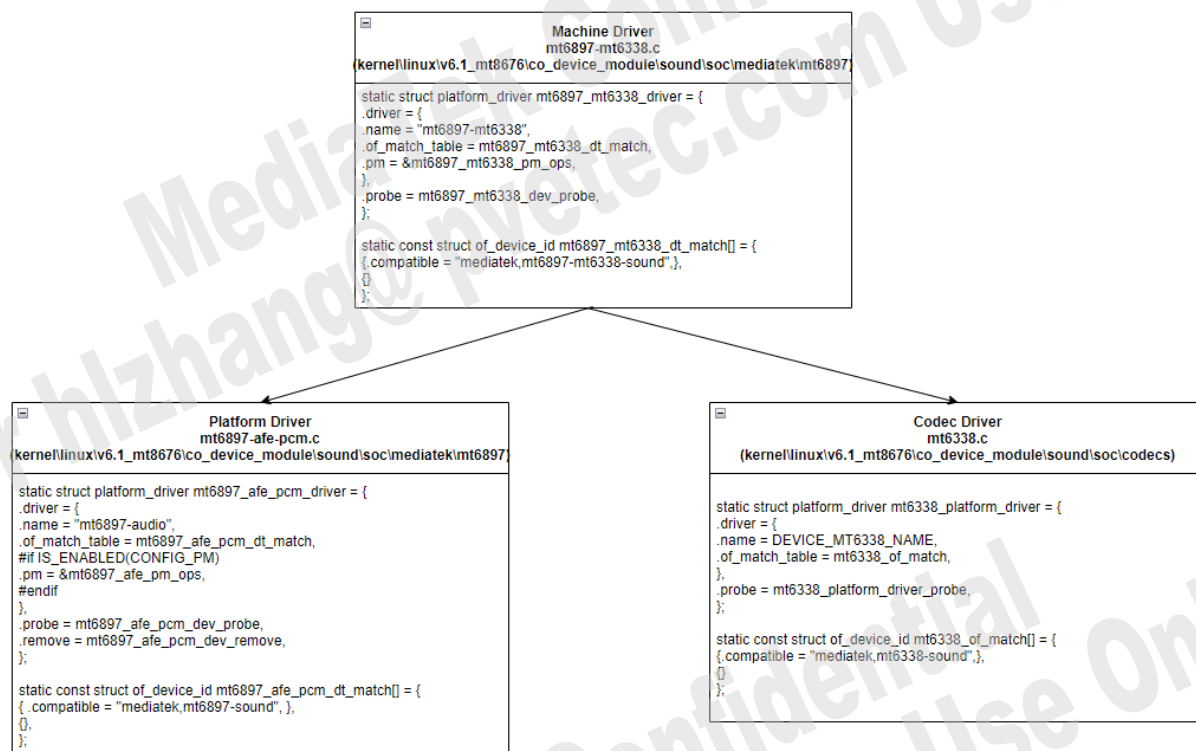


Figure 1-5. Audio driver components

These components of the Audio Driver are initialized sequentially during the Kernel initialization process after the system starts. Once the system startup is complete, you can view the corresponding device information in the adb shell console interface:

```

sh-3.2# cat /proc/asound/cards
 0 [mt6897:mt6338 ]: mt6897-mt6338 - mt6897-mt6338 machine driver
mt6897-mt6338

sh-3.2# ls /sys/kernel/debug/asoc/mt6897-mt6338
11050000.mt6897-afe-pcm Capture_Mono_1 Playback_1
18830000.mtk-btcsd-snd Capture_Mono_2 Playback_11
Capture_1 Hostless_ADDA_DL_I2S_OUT Playback_12
Capture_10 Hostless_BT Playback_13
Capture_11 Hostless_FM Playback_2
Capture_12 Hostless_HW_Gain_AAudio Playback_25
Capture_13 Hostless_HW_SRC_1_IN Playback_3
Capture_2 Hostless_HW_SRC_1_OUT Playback_4
Capture_3 Hostless_HW_SRC_2_IN Playback_5
Capture_4 Hostless_HW_SRC_2_OUT Playback_6
Capture_5 Hostless_HW_SRC_3_IN Playback_7
Capture_6 Hostless_HW_SRC_3_OUT Playback_8
Capture_7 Hostless_LPEK Playback_9
Capture_8 Hostless_SRC_1 Playback_HDMI
Capture_9 Hostless_SRC_3 dapm
Capture_ETDM_In0 Hostless_SRC_AAudio dapm_pop_time
Capture_ETDM_In1 Hostless_SRC_Bargein mt-soc-offload-common
Capture_ETDM_In2 Hostless_Speech mt6338-sound
Capture_ETDM_In4 Hostless_Sph_Echo_Ref snd-audio-dsp
Capture_ETDM_In6 Hostless_Spk_Init snd-scp-ultra

sh-3.2#
sh-3.2# ls /sys/kernel/debug/asoc/
components dais mt6897-mt6338
sh-3.2# cat /sys/kernel/debug/asoc/components
mt6338-sound codec driver
11050000.mt6897-afe-pcm platform driver
mt-soc-offload-common
snd-audio-dsp
snd-scp-ultra
18830000.mtk-btcsd-snd
snd-soc-dummy
snd-soc-dummy

```

Figure 1-6. Audio devices

According to the DPCM architecture, the ALSA driver virtualizes memory interfaces as Front End (FE) devices and external hardware interfaces as Back End (BE) devices. FE and BE are connected through connection interfaces to form an audio path, which can be used for audio data transmission. Since both FE and BE are audio interfaces within the MT8675 SoC, the ALSA Driver treats them uniformly as CPU DAI. The further backend PMIC or external DSP is considered the Codec DAI.

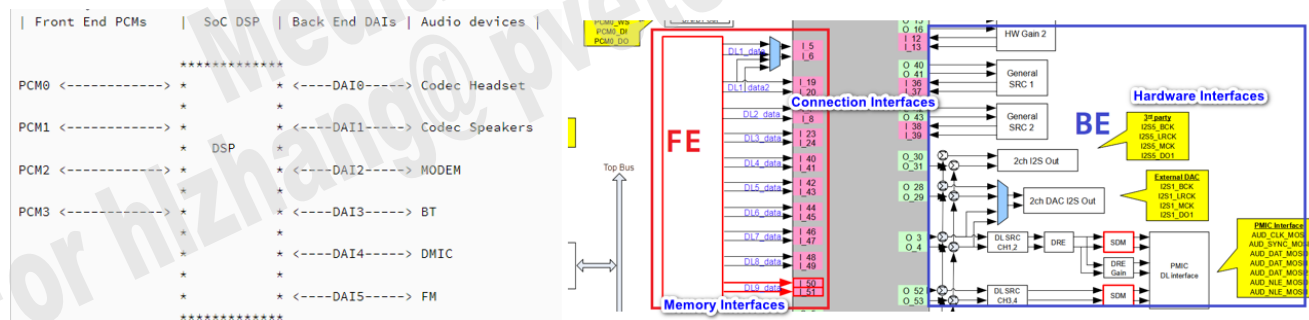
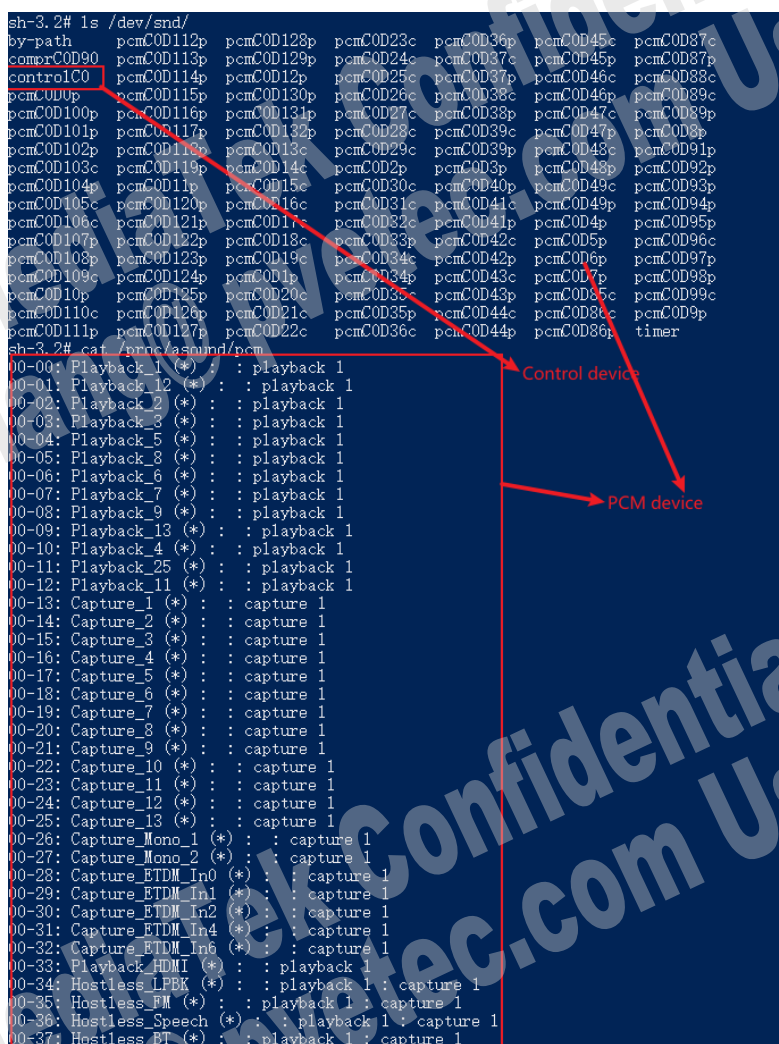


Figure 1-7. Audio CPU DAI

The ALSA driver constructs a series of Control/Mixer APIs to control the DAI and establish the audio path. The devices used to control these APIs are known as Control devices. Similarly, the devices used to control specific FEs and read/write data are known as PCM devices.

Figure 1-8 shows the Control device nodes and PCM device nodes for the MT8676 platform:



```

sh-3.2# ls /dev/snd/
by-path      pcmC0D112p  pcmC0D128p  pcmC0D23c  pcmC0D36p  pcmC0D45c  pcmC0D87c
comonC0D90  pcmC0D113p  pcmC0D129p  pcmC0D24c  pcmC0D37c  pcmC0D45p  pcmC0D87p
controlC0    pcmC0D114p  pcmC0D12p   pcmC0D25c  pcmC0D37p  pcmC0D46c  pcmC0D88c
pcmC0D0p     pcmC0D115p  pcmC0D130p  pcmC0D26c  pcmC0D38c  pcmC0D46p  pcmC0D89c
pcmC0D100p   pcmC0D116p  pcmC0D131p  pcmC0D27c  pcmC0D38p  pcmC0D47c  pcmC0D89p
pcmC0D101p   pcmC0D117p  pcmC0D132p  pcmC0D28c  pcmC0D39c  pcmC0D47p  pcmC0D8p
pcmC0D102p   pcmC0D118p  pcmC0D133p  pcmC0D29c  pcmC0D39p  pcmC0D48c  pcmC0D91p
pcmC0D103c   pcmC0D119p  pcmC0D14c   pcmC0D2p   pcmC0D3p   pcmC0D48p  pcmC0D92p
pcmC0D104p   pcmC0D11p   pcmC0D15c   pcmC0D30c  pcmC0D40p  pcmC0D49c  pcmC0D93p
pcmC0D105c   pcmC0D120p  pcmC0D16c   pcmC0D31c  pcmC0D41c  pcmC0D49p  pcmC0D94p
pcmC0D106c   pcmC0D121p  pcmC0D17c   pcmC0D32c  pcmC0D41p  pcmC0D4p   pcmC0D95p
pcmC0D107p   pcmC0D122p  pcmC0D18c   pcmC0D33p  pcmC0D42c  pcmC0D5p   pcmC0D96c
pcmC0D108p   pcmC0D123p  pcmC0D19c   pcmC0D34c  pcmC0D42p  pcmC0D6p   pcmC0D97p
pcmC0D109c   pcmC0D124p  pcmC0D1p    pcmC0D34p  pcmC0D43c  pcmC0D7p   pcmC0D98p
pcmC0D10p    pcmC0D125p  pcmC0D20c   pcmC0D35c  pcmC0D43p  pcmC0D85c  pcmC0D99c
pcmC0D110c   pcmC0D126p  pcmC0D21c   pcmC0D35p  pcmC0D44c  pcmC0D86c  pcmC0D9p
pcmC0D111p   pcmC0D127p  pcmC0D22c   pcmC0D36c  pcmC0D44p  pcmC0D86p  timer
sh-3.2# cat /proc/asound/pcm
00-00: Playback_1 (*) : : playback 1
00-01: Playback_12 (*) : : playback 1
00-02: Playback_2 (*) : : playback 1
00-03: Playback_3 (*) : : playback 1
00-04: Playback_5 (*) : : playback 1
00-05: Playback_8 (*) : : playback 1
00-06: Playback_6 (*) : : playback 1
00-07: Playback_7 (*) : : playback 1
00-08: Playback_9 (*) : : playback 1
00-09: Playback_13 (*) : : playback 1
00-10: Playback_4 (*) : : playback 1
00-11: Playback_25 (*) : : playback 1
00-12: Playback_11 (*) : : playback 1
00-13: Capture_1 (*) : : capture 1
00-14: Capture_2 (*) : : capture 1
00-15: Capture_3 (*) : : capture 1
00-16: Capture_4 (*) : : capture 1
00-17: Capture_5 (*) : : capture 1
00-18: Capture_6 (*) : : capture 1
00-19: Capture_7 (*) : : capture 1
00-20: Capture_8 (*) : : capture 1
00-21: Capture_9 (*) : : capture 1
00-22: Capture_10 (*) : : capture 1
00-23: Capture_11 (*) : : capture 1
00-24: Capture_12 (*) : : capture 1
00-25: Capture_13 (*) : : capture 1
00-26: Capture_Mono_1 (*) : : capture 1
00-27: Capture_Mono_2 (*) : : capture 1
00-28: Capture_ETDM_In0 (*) : : capture 1
00-29: Capture_ETDM_In1 (*) : : capture 1
00-30: Capture_ETDM_In2 (*) : : capture 1
00-31: Capture_ETDM_In4 (*) : : capture 1
00-32: Capture_ETDM_In6 (*) : : capture 1
00-33: Playback_HDMI (*) : : playback 1
00-34: Hostless_LPBK (*) : : playback 1 : capture 1
00-35: Hostless_FM (*) : : playback 1 : capture 1
00-36: Hostless_Speech (*) : : playback 1 : capture 1
00-37: Hostless_BT (*) : : playback 1 : capture 1

```

Figure 1-8. DPCM devices

The correspondence between PCM nodes and AFE is illustrated in Figure 1-9:

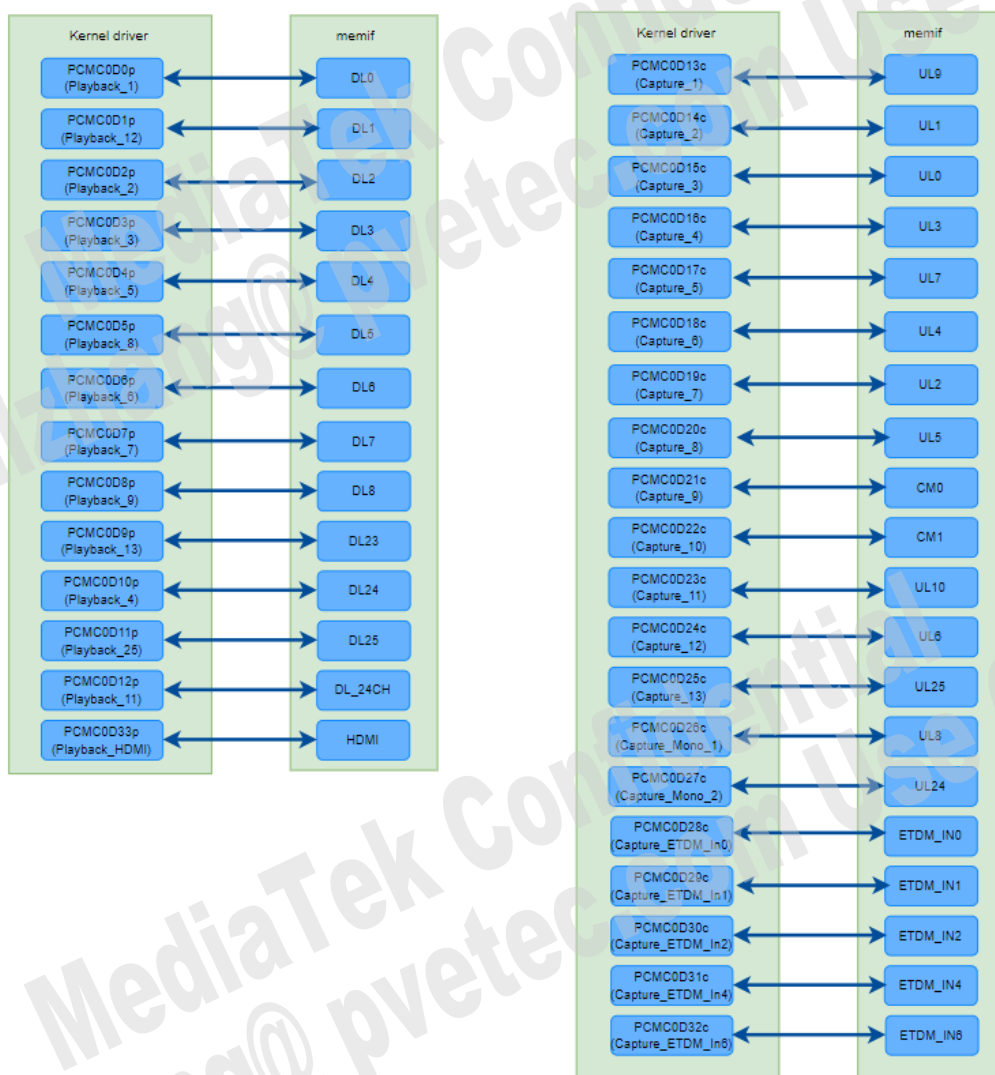


Figure 1-9. Playback & capture devices

In playback scenarios, such as playing music, the application layer writes audio data to the Playback PCM node, which is then transmitted to the backend speakers or amplifiers to produce sound. Conversely, in capture scenarios, such as recording, the application layer reads audio data from the Capture PCM node, obtaining the audio data input from external devices like microphones.

1.3 Configuration/Customization Guideline

1.3.1 Kernel DTS

The configuration of the ALSA driver is primarily completed in the DTS (Device Tree Source) files, which include the configuration of device nodes for drivers related to Audio, such as Machine, Platform, and Codec. These configurations encompass register addresses, memory addresses, interrupt numbers, and other parameters.

The DTS configuration files for the MT8676 are located at:

src/kernel/linux/v6.1_mt8676/co_device_module/arch/arm64/boot/dts/mediatek/. The main configuration files are as follows:

```
mt6897.dts
auto8676p1_64.dts
```

The corresponding configurations for Machine, Platform, and Codec are shown as [Figure 1-10](#):

```

machine driver
sound: sound {
    compatible = "mediatek,mt6897-mt6338-sound";
    /* mediatek_snd_audio_dsp = <snd_audio_dsp>; */
    mediatek_headset_codec = <mt6338_accdet>;
    mediatek_platform = <afe>;
    mediatek_ipm = <1>; /* 0: old version; 1: IPM2.0 */
};

platform driver
afe: mt6897-afe-pcm@11050000 {
    compatible = "mediatek,mt6897-sound";
    reg = <0 0x11050000 0 0x9000>;
    interrupts = <GIC_SPI 257 IRQ_TYPE_LEVEL_HIGH 0>;
    topckgen = <topckgen_clk>;
    apmixedsys = <apmixedsys_clk>;
    infracfg = <infracfg_a0_clk>;
    power-domains = <scpsys MT6897_POWER_DOMAIN_AUDIO>;
    clocks = <afe_clk CLK_AFE_AUDIO_HOPPING>,
        <afe_clk CLK_AFE_AUDIO_F26M>,
        <afe_clk CLK_AFE_DLO_DAC>,
        <afe_clk CLK_AFE_DLO_DAC_HIRES>,
        <afe_clk CLK_AFE_DLO_PREDIS>,
        <afe_clk CLK_AFE_DL1_DAC>,
        <afe_clk CLK_AFE_DL1_DAC_HIRES>,
        <afe_clk CLK_AFE_DL1_PREDIS>,
        <afe_clk CLK_AFE_UL0_ADC>,
        <afe_clk CLK_AFE_UL0_ADC_HIRES>,
        <afe_clk CLK_AFE_UL1_ADC>,
        <afe_clk CLK_AFE_UL1_ADC_HIRES>,
        <afe_clk CLK_AFE_APLL1>,
        <afe_clk CLK_AFE_APLL2>,
        <afe_clk CLK_AFE_APLL_TUNER1>,
        <afe_clk CLK_AFE_APLL_TUNER2>,
        <topckgen_clk CLK_TOP_AUD_INBUS_SEL>,
        <topckgen_clk CLK_TOP_MAINPLL_D4_D4>,
        <topckgen_clk CLK_TOP_AUD_1_SEL>,
        <topckgen_clk CLK_TOP_APLL1>,
        <topckgen_clk CLK_TOP_APLL2>;
};

codec driver
mt6338_sound: mt6338-sound {
    compatible = "mediatek,mt6338-sound";
    mediatek_dmic-mode = <0>;
    mediatek_mic-type-0 = <3>;
    mediatek_mic-type-1 = <3>;
    mediatek_mic-type-2 = <3>;
    io-channels = <mt6338_auxadc AUXADC_HPOFS_CAL>;
    io-channel-names = "pmic_hpofs_cal";
    nvmem = <mt6338_efuse>;
    nvmem-names = "pmic-hp-efuse";
};

```

Figure 1-10. MTK Audio DTS

When the Kernel initializes, the various components of the Audio Driver can obtain the necessary parameters from the DTS configuration, such as register addresses, memory addresses, clock configurations, and GPIO settings. During the use of the Audio Driver, these parameters are used to configure the audio hardware and establish the audio path. therefore, when developing on the customer platform, it is essential to customize the corresponding DTS configuration according to the actual hardware conditions.

1.3.2 Audio Device Configuration

Before an Audio APP can perform read/write operations on a specific PCM device node, it needs to configure some mixer/control settings to open the audio path or enable certain audio hardware interfaces.

For example, to play music on the MT8676 reference board, the following steps are used to establish the audio path through the Playback_2 node, connecting DL2 → ADDA_DL → PMIC 6338 → Speaker:

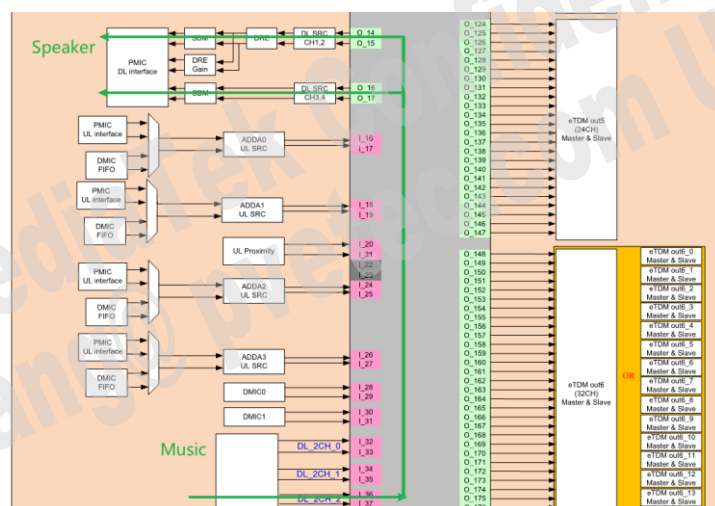


Figure 1-11. Audio path for playing music

The kcontrol settings are as follows:

- Connects DL2 to PMIC

```
'ADDA_DL_CH1 DL2_CH1' 1
'ADDA_DL_CH2 DL2_CH2' 1
'ADDA_DL_CH3 DL2_CH1' 1
'ADDA_DL_CH4 DL2_CH2' 1
```

- Connects PMIC to Speaker:

```
'HPL Mux' 'Audio Playback'
'HPR Mux' 'Audio Playback'
```

1.4 Frequently Asked Questions/Troubleshooting

1.4.1 How to Test Audio Driver?

When the Audio Driver is ready, you can use the built-in Yocto tools aplay, arecord, and amixer for manual testing. amixer is used to connect the audio path, aplay is used for playing sound, and arecord is used for recording from the MIC. For example, to play audio through Playback_2, the command line is as follows:

```
adb shell
amixer cset name='ADDA_DL_CH1 DL2_CH1' 1
amixer cset name='ADDA_DL_CH2 DL2_CH2' 1
amixer cset name='ADDA_DL_CH3 DL2_CH1' 1
amixer cset name='ADDA_DL_CH4 DL2_CH2' 1
amixer cset name='HPL Mux' 'Audio Playback'
amixer cset name='HPR Mux' 'Audio Playback'
aplay -Dhw:0,2 /tmp/test_music.wav &
```

1.4.2 How to Enable Audio Log?

When developing or troubleshooting bugs, you need to enable the Audio Driver log to pinpoint issues.

Enable the Audio Driver log: `echo 7 > /proc/sys/kernel/printk`

After setting this, you can use adb shell logcat to view the output logs.

Exhibit 1 Terms and Conditions

Your access to and use of this document and the information contained herein (collectively this “Document”) is subject to your (including the corporation or other legal entity you represent, collectively “You”) acceptance of the terms and conditions set forth below (“T&C”). By using, accessing or downloading this Document, You are accepting the T&C and agree to be bound by the T&C. If You don’t agree to the T&C, You may not use this Document and shall immediately destroy any copy thereof.

This Document contains information that is confidential and proprietary to MediaTek Inc. and/or its affiliates (collectively “MediaTek”) or its licensors and is provided solely for Your internal use with MediaTek’s chipset(s) described in this Document and shall not be used for any other purposes (including but not limited to identifying or providing evidence to support any potential patent infringement claim against MediaTek or any of MediaTek’s suppliers and/or direct or indirect customers). Unauthorized use or disclosure of the information contained herein is prohibited. You agree to indemnify MediaTek for any loss or damages suffered by MediaTek for Your unauthorized use or disclosure of this Document, in whole or in part.

MediaTek and its licensors retain titles and all ownership rights in and to this Document and no license (express or implied, by estoppels or otherwise) to any intellectual propriety rights is granted hereunder. This Document is subject to change without further notification. MediaTek does not assume any responsibility arising out of or in connection with any use of, or reliance on, this Document, and specifically disclaims any and all liability, including, without limitation, consequential or incidental damages.

THIS DOCUMENT AND ANY OTHER MATERIALS OR TECHNICAL SUPPORT PROVIDED BY MEDIATEK IN CONNECTION WITH THIS DOCUMENT, IF ANY, ARE PROVIDED “AS IS” WITHOUT WARRANTY OF ANY KIND, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE. MEDIATEK SPECIFICALLY DISCLAIMS ALL WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, COMPLETENESS OR ACCURACY AND ALL WARRANTIES ARISING OUT OF TRADE USAGE OR OUT OF A COURSE OF DEALING OR COURSE OF PERFORMANCE. MEDIATEK SHALL NOT BE RESPONSIBLE FOR ANY MEDIATEK DELIVERABLES MADE TO MEET YOUR SPECIFICATIONS OR TO CONFORM TO A PARTICULAR STANDARD OR OPEN FORUM.

Without limiting the generality of the foregoing, MediaTek makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does MediaTek assume any liability arising out of the application or use of any product, circuit or software. You agree that You are solely responsible for the designing, validating and testing Your product incorporating MediaTek’s product and ensure such product meets applicable standards and any safety, security or other requirements.

The above T&C and all acts in connection with the T&C or this Document shall be governed, construed and interpreted in accordance with the laws of Taiwan, without giving effect to the principles of conflicts of law.